THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE
CS-4031 Compiler Construction — Fall 2026

Lecture 8
The Landscape of Parsing; Context-Free Grammars and Derivations; Grammar Engineering: Left Factoring and Left Recursion

CLO(s) covered:	CLO 1, CLO 2
Dragon Book [DB]:	§4.1 (Introduction to Syntax Analysis: top-down vs. bottom-up), §4.2 (Context-Free Grammars: derivations, parse trees), §4.3.3 (Elimination of Left Recursion), §4.3.4 (Left Factoring)
Other references:	[ET] Engineering a Compiler, §3.1–3.2; [AP] Modern Compiler Implementation, Ch. 3

Lecture Roadmap

1. A **bird's-eye map of parsing**: every parsing algorithm this course will ever name falls under either **top-down** or **bottom-up** construction of a parse tree, and every algorithm's one-line "key feature" is worth knowing before we build any of them in depth.
2. **Context-free grammars (CFGs)**: the formalism parsing needs, precisely because Lecture **??** already showed regular expressions cannot express nested structure.
3. **Derivations and parse trees**: what it formally means to "generate" a string from a grammar, worked through on one running example.
4. **Grammar engineering, part 1 – Left Factoring**: why sharing a common prefix between productions is a real problem for a predictive parser, and how to remove it.
5. **Grammar engineering, part 2 – Left Recursion**: a *different*, more severe problem specifically for top-down parsers, and how to remove it – both the simple (direct) and general (indirect) cases.

This is deliberately a *first* lecture on parsing: a map of the territory and the grammar hygiene every later algorithm assumes has already been done, not yet any actual parsing algorithm. The lectures ahead build the leaves of today's tree one at a time, in depth.

8.1 The Landscape of Parsing

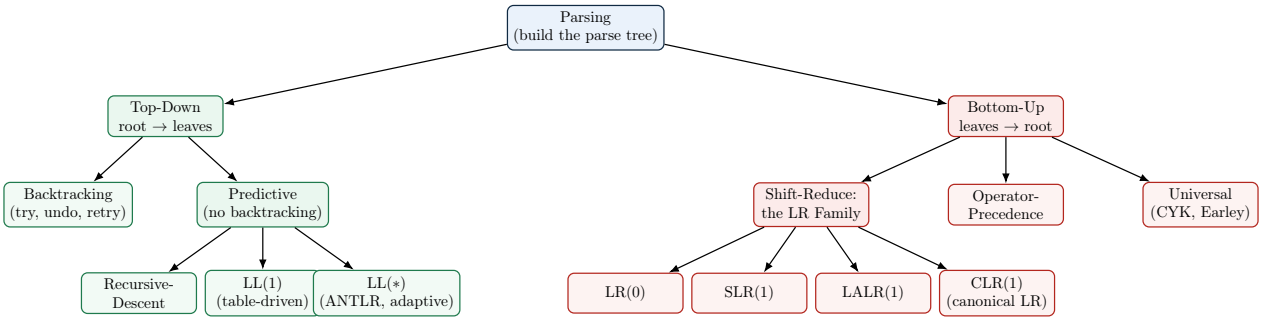
[DB] Lecture **??**'s phase pipeline (§**??**) named **syntax analysis** as the phase right after lexical analysis: it takes the token stream and organizes it into a **parse tree** (or **syntax tree**) reflecting the program's grammatical structure. Every algorithm that does this job falls into exactly one of two families, distinguished by *which end of the parse tree it builds first.*

Definition: Top-Down vs. Bottom-Up Parsing

- A **top-down** parser builds the parse tree starting at the **root** (the grammar's start symbol) and works *downward* toward the leaves (the actual input tokens), at every point guessing/deriving which production to expand next so that the eventual leaves match the input.
- A **bottom-up** parser builds the parse tree starting at the **leaves** (the input tokens themselves) and works *upward*, repeatedly recognizing a piece of the input that matches the right-hand side of some production and collapsing it into that production's left-hand side,

until only the start symbol remains.

Both families are guaranteed to produce the *same* parse tree for an unambiguous grammar on a given input – they differ only in the *order* in which that one tree gets constructed, which turns out to have major consequences for which grammars each family can handle and how each is implemented.



Family	Algorithm	One-Line Key Feature
Top-down	Backtracking	Tries a production, and if it leads to a dead end, undoes the choice and tries the next one – simple to describe, but can cost exponential time in the worst case; essentially never used in production compilers.
Top-down	Recursive-Descent	One hand-written function per nonterminal, calling other such functions to match the right-hand side – simple, flexible, easy to add custom error messages to, but *requires* a grammar with no left recursion and no unresolved left factors (§8.4–§8.5).
Top-down	LL(1)	A *mechanically constructed* parsing table, built from FIRST/FOLLOW sets, driven by a generic stack-based simulation loop – "LL" = read input Left-to-right, produce a Leftmost derivation, using 1 token of lookahead.
Top-down	LL(∗)	A modern relative of LL(1) that adaptively looks ahead as many tokens as needed (not just one) to decide between productions – what ANTLR4 actually builds; more grammars accepted than plain LL(1), at the cost of a more complex parser generator.
Bottom-up	LR(0)	The base LR automaton, built from "items" with *no* lookahead at all – rarely used directly (it accepts very few grammars), but it is the foundation SLR(1), LALR(1), and CLR(1) all extend with progressively richer lookahead.
Bottom-up	SLR(1)	"Simple LR": the smallest, easiest-to-build LR table, using the LR(0) automaton plus FOLLOW sets to decide when to reduce – but the weakest lookahead-equipped member of the LR family, rejecting some genuinely unambiguous grammars.
Bottom-up	LALR(1)	Merges canonical LR(1) states that share the same core – same (small) table size as SLR(1), but strictly more grammars accepted; the industry-standard algorithm (what Yacc/Bison actually build).
Bottom-up	CLR(1)	"Canonical LR(1)": keeps every LR(1) state distinct, with full per-state lookahead – the most powerful of the three, accepting the largest class of grammars, but with tables that can be dramatically larger, which is exactly why LALR(1) is preferred in practice.
Bottom-up	Operator-Precedence	An older, simpler technique that only ranks operators against each other (no full grammar automaton needed) – easy to hand-build for expression grammars, but far less general than the LR family; largely superseded by LR parsing today.
Bottom-up	Universal (CYK, Earley)	General-purpose algorithms that can parse *any* context-free grammar (even ambiguous ones), using dynamic programming – CYK needs the grammar in a normal form and runs in $O(n^3)$; Earley handles arbitrary CFGs directly. Too slow for everyday compiler use, but standard in NLP parsing and grammar-tooling contexts where generality matters more than raw speed.

Exam Focus

Memorize the shape of this tree, not just the list of names: **every** bottom-up algorithm in this course is a member of the **shift-reduce** family (they all maintain a stack and decide, at each step, whether to *shift* the next input symbol onto the stack or *reduce* the stack's top symbols using a production) – SLR(1), LALR(1), and CLR(1) differ *only* in how much lookahead information their tables encode, not in the shift-reduce mechanism itself. This is exactly the kind of "classify this algorithm" question that shows up on exams: given an algorithm's name, you should be able to place it correctly on this tree without hesitation.

What's a Preview, and What's a Full Build

Today you are seeing every leaf of this tree *by name only* – a bird's-eye key feature, nothing more. The lectures ahead build several of these leaves in full mechanical detail (how to construct a FIRST/FOLLOW-based LL(1) table; how to build an LR(0) automaton and extend it to SLR(1), then LALR(1), with a brief look at canonical LR(1)) – but before any of that construction can begin, every grammar first needs two specific problems fixed. That is the second half of today's lecture.

8.2 Context-Free Grammars

[DB] Lecture **??**'s §**??** already proved the gap this section fills: regular expressions have no way to *count* an unbounded quantity, which is exactly why balanced parentheses – and, by the same argument, the nested structure of every real programming language's expressions, blocks, and function calls – cannot be expressed with a regular expression. A **context-free grammar (CFG)** is the more powerful formalism that *can* express this nesting, and it is the formalism every parser in §8.1's tree is built to process.

Definition: Context-Free Grammar

A context-free grammar is a 4-tuple (N, T, P, S) where:

- N is a finite set of **nonterminals** – syntactic categories like `expr` or `stmt` that stand for a set of strings, not any one fixed string;
- T is a finite set of **terminals** – the actual tokens coming out of the lexer (§**??**), disjoint from N;
- P is a finite set of **productions**, each of the form $A \to \alpha$ where $A \in N$ and $\alpha \in (N \cup T)^*$ – "nonterminal A can be replaced by the string α"; several productions for the same A are usually written together as $A \to \alpha_1 \mid \alpha_2 \mid \ldots$;
- $S \in N$ is the **start symbol**, the one nonterminal every derivation begins from.

Example: A Running Example: Simple Arithmetic Expressions

This grammar for `+`, `*`, parentheses, and `id` will be reused for every worked example in the rest of this lecture:

$$E \to E + T \mid T$$
$$T \to T * F \mid F$$
$$F \to (E) \mid \texttt{id}$$

Here $N = \{E, T, F\}$, $T_{\text{terminals}} = \{\texttt{+}, \texttt{*}, (,), \texttt{id}\}$, and the start symbol is E. Notice the grammar already encodes operator precedence structurally: T (a "term") can only ever appear as a `+`-separated piece of E, and F (a "factor") can only appear as a `*`-separated piece of T – multiplication is forced to bind tighter than addition purely by *which nonterminal contains which*, with no explicit precedence numbers anywhere.

8.3 Derivations and Parse Trees

Definition: Derivation

A **derivation** is a sequence of production applications, starting from the start symbol S, that rewrites one nonterminal at a time until only terminals remain. Each intermediate string (a mix of terminals and nonterminals) is a **sentential form**. A **leftmost derivation** always rewrites the *leftmost* remaining nonterminal at every step; a **rightmost derivation** always rewrites the

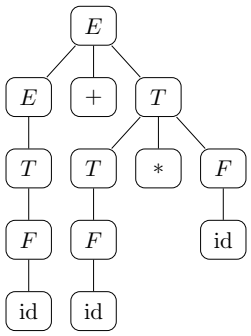
rightmost one. (Both are valid, equally correct ways to generate the *same* string – they simply commit to a fixed order so the derivation is unambiguous to describe, and different parser families naturally correspond to one or the other, as §8.1's LL/LR naming already hinted.)

Example: Worked Example: Leftmost Derivation of `id + id * id`

Using §8.2's expression grammar:

$$
\begin{aligned}
E &\Rightarrow E + T && (E \to E + T)\\
&\Rightarrow T + T && (E \to T)\\
&\Rightarrow F + T && (T \to F)\\
&\Rightarrow \texttt{id} + T && (F \to \texttt{id})\\
&\Rightarrow \texttt{id} + T * F && (T \to T * F)\\
&\Rightarrow \texttt{id} + F * F && (T \to F)\\
&\Rightarrow \texttt{id} + \texttt{id} * F && (F \to \texttt{id})\\
&\Rightarrow \texttt{id} + \texttt{id} * \texttt{id} && (F \to \texttt{id})
\end{aligned}
$$

At every step, the *leftmost* nonterminal in the current sentential form is the one rewritten – notice how the parenthesization implied by precedence (multiplication grouped under one T) falls straight out of *which* nonterminal gets expanded, never out of any explicit precedence rule.



The parse tree for `id + id * id`: exactly the tree the leftmost derivation above builds, one production per internal node. Reading the leaves left to right reproduces the original input string – this "yield" property holds for any *valid parse tree, top-down or bottom-up.*

Exam Focus

A single parse tree corresponds to *many* different derivation orders (leftmost, rightmost, or anything in between) – the tree is the ground truth; a derivation is just one particular order of announcing it. Do not confuse "two different derivations" with "two different parse trees": the former can happen for a perfectly ordinary, unambiguous grammar (swap which nonterminal you expand first); the latter – genuinely different *trees* for the *same* string – is what **ambiguity** means, and is the subject of the next lecture, not this one.

8.4 Grammar Engineering, Part 1: Left Factoring

[DB] A top-down parser – whether hand-written recursive descent or a generated LL(1) table – has to decide *which production to use* for the current nonterminal by looking at only a small, fixed amount of upcoming input (for LL(1), exactly *one* token). That decision becomes impossible whenever two productions for the same nonterminal start the same way.

Definition: Left Factoring

A grammar has a **left factor** (a common prefix) when some nonterminal A has two or more productions $A \to \alpha\beta_1 \mid \alpha\beta_2 \mid \ldots$ that begin with the same nonempty string α. **Left factoring** is the grammar transformation that rewrites these into $A \to \alpha A'$ and $A' \to \beta_1 \mid \beta_2 \mid \ldots$ – deferring the decision of *which* β_i until after α has already been matched and a new nonterminal A' is reached, at which point enough input has been consumed to decide correctly.

Why Common Prefixes Arise, and Why They're a Real Problem

Shared prefixes are not a rare, contrived situation – they show up naturally wherever a language offers an *optional trailing clause* on top of a shorter, complete construct. DB's classic example: an `if` statement with an *optional* `else` – `if expr then stmt` is already a complete statement on its own, but `if expr then stmt else stmt` is also legal, and the two productions share the entire prefix `if expr then stmt`. A predictive parser reading left to right cannot know, right after matching `then stmt`, whether it should be using the first production or the second – both are still equally plausible until the *next* token (`else`, or something else) actually arrives. Left uncorrected, the parser is left with exactly two bad options, and both cost real time on *every single* parse, not just pathological ones:

1. **Backtracking** – guess a production, and if a later mismatch reveals the guess was wrong, undo everything done so far and try the next production. This can require re-scanning already-processed input, repeatedly, and in the worst case costs exponential time (§8.1's "Backtracking" row).
2. **Extra lookahead** – peek further ahead than one token to decide, which not only slows down *every* parsing step (not just the ambiguous ones) but can require arbitrarily much lookahead for some grammars, defeating the whole point of a simple, fixed-lookahead table-driven parser.

Left factoring sidesteps both costs entirely: after the transformation, one token of lookahead is always enough, because the parser is never asked to choose between competing productions until they have actually diverged.

Example: Worked Example 1: The Classic `if`/`else`

Before:

$$\begin{aligned} \texttt{stmt} \to\ & \texttt{if}\ \textit{expr}\ \texttt{then}\ \textit{stmt} \\ \mid\ & \texttt{if}\ \textit{expr}\ \texttt{then}\ \textit{stmt}\ \texttt{else}\ \textit{stmt} \\ \mid\ & \texttt{other} \end{aligned}$$

After (common prefix `if` *expr* `then` *stmt* factored out):

$$\begin{aligned} \texttt{stmt} &\to \texttt{if}\ \textit{expr}\ \texttt{then}\ \textit{stmt}\ \textit{stmt}' \mid \texttt{other} \\ \textit{stmt}' &\to \texttt{else}\ \textit{stmt} \mid \varepsilon \end{aligned}$$

Every string the original grammar could generate, the new grammar still generates – left factoring changes *how* a string is derived, never *which* strings are in the language. A predictive parser can now decide correctly: after matching `if expr then stmt`, it peeks at the next token; if it's `else`, use $\textit{stmt}' \to \texttt{else}\ \textit{stmt}$; otherwise, use $\textit{stmt}' \to \varepsilon$ and move on. (This is also exactly the classic "dangling else" ambiguity in disguise – the next lecture returns to it from the ambiguity angle specifically.)

Example: Worked Example 2: A Smaller, Purely Mechanical Case

Before: $A \to \mathtt{a}A_1 \mid \mathtt{a}A_2 \mid \mathtt{a}A_3$

After: $A \to \mathtt{a}A' \qquad A' \to A_1 \mid A_2 \mid A_3$

The common prefix here is the single terminal `a`; factoring it out leaves the new nonterminal A' to decide among A_1, A_2, A_3 using whatever token comes *after* the `a` – exactly one token further than the original grammar could safely commit to.

Exam Focus

Left factoring sometimes has to be applied **more than once**: factoring out one common prefix can expose a *new* common prefix one level deeper (e.g. if A_1 and A_2 above themselves both started with the same symbol). DB's algorithm is understood to repeat until no nonterminal has any left-factorable productions left – checking once and stopping is a common exam and implementation mistake.

8.5 Grammar Engineering, Part 2: Left Recursion

[DB] A second, unrelated grammar property is just as disqualifying for top-down parsing – but for a completely different, more severe reason than left factoring's slowdown.

Definition: Left Recursion

A grammar is **immediately left-recursive** if some nonterminal A has a production of the form $A \to A\alpha$ – A's own right-hand side begins with A itself. More generally, a grammar is (**indirectly**) left-recursive if $A \Rightarrow^+ A\alpha$ for *some* sequence of production applications, for some string α – A eventually derives something beginning with A again, possibly by passing through several *other* nonterminals first, not just directly.

Why Left Recursion Is Worse Than a Slowdown: Infinite Loops

Recall §8.1's recursive-descent row: a top-down parser calls one function per nonterminal, and to parse A it calls `parseA()`. If $A \to A\alpha$ is one of A's productions, then `parseA()`'s very first action – *before consuming a single input token* – is to call `parseA()` again, to match that leading A. That inner call does exactly the same thing, calling `parseA()` once more, and so on forever: the recursion never bottoms out, because no input is ever consumed to make progress toward a base case. The concrete result is unbounded recursion and a stack overflow, not merely a slow parse. This is the precise, mechanical reason left recursion and left factoring are *both* grammar-engineering steps every top-down technique requires, yet are fundamentally different kinds of problems:

	Left Factoring (§8.4)	**Left Recursion (here)**
Symptom	Two or more productions for the same nonterminal share a common prefix	Some nonterminal, directly or indirectly, has a production whose right-hand side starts with itself
What goes wrong for a top-down parser	Cannot choose which production to use with only one token of lookahead	Cannot make *any* progress at all before recursing again
Consequence if left unfixed	Forced into backtracking or extra lookahead – a real, but bounded, performance cost	Unbounded recursion – the parser **never terminates** (stack overflow), regardless of the input
Severity	A performance / parser-design problem	A **correctness** problem – the parser is simply broken for this grammar

Exam Focus

A very common exam confusion: describing left recursion as "making the parser slow," the same language used for left factoring. Be precise – left recursion does not slow a top-down parser down, it makes it **not terminate at all** for a naive implementation. Bottom-up (LR-family) parsers, by contrast, have *no* problem with left recursion at all – in fact left-recursive grammars are often *preferred* for LR parsing, since they keep the parser's stack bounded where an equivalent right-recursive grammar would not. Left recursion is specifically a **top-down** problem, not a universal one.

8.5.1 Eliminating Immediate Left Recursion

The Standard Transformation

Given $A \to A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$, where none of the β_i themselves start with A, rewrite as:

$$A \to \beta_1 A' \mid \beta_2 A' \mid \cdots \mid \beta_n A' \qquad A' \to \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_m A' \mid \varepsilon$$

In words: every alternative that *doesn't* start with A (a β_i) becomes a starting point, immediately followed by a new nonterminal A'; every alternative that *did* start with A (an α_i) becomes a *tail*, hung off A' instead, with A' itself allowed to be empty. The recursion has not disappeared – it has simply been converted from *left*-recursion into *right*-recursion (every recursive call to A' now comes *after* consuming some real input), which top-down parsing has no trouble with at all.

Example: Worked Example: Removing Left Recursion From the Expression Grammar

Applying the transformation to §8.2's running example, nonterminal by nonterminal:

$$\begin{array}{ll} \texttt{Before:} & E \to E + T \mid T \qquad (\alpha_1 = +T,\ \beta_1 = T) \\ \texttt{After:} & E \to TE' \qquad E' \to +TE' \mid \varepsilon \\ \\ \texttt{Before:} & T \to T * F \mid F \qquad (\alpha_1 = *F,\ \beta_1 = F) \\ \texttt{After:} & T \to FT' \qquad T' \to *FT' \mid \varepsilon \\ \\ \texttt{Unchanged:} & F \to (E) \mid \texttt{id} \qquad (F \text{ was never left-recursive}) \end{array}$$

The resulting grammar generates *exactly* the same language as before, but a `parseE()` function built from it now consumes at least one token (via `parseT()`) before ever calling `parseE'()` – the infinite-regress problem from the coreidea box above is gone by construction.

8.5.2 Eliminating Indirect (General) Left Recursion

[DB] Left recursion can hide across *several* nonterminals, with no single production obviously guilty on its own – A may not start with A anywhere in the grammar as written, yet still derive something starting with A once other nonterminals are expanded. The general algorithm handles this in exactly the two steps the immediate case already gave us, just with one extra step in front: **first turn the indirect recursion into direct (immediate) recursion by substitution, then eliminate that immediate recursion exactly as in §8.5.1.**

DB Algorithm 4.19 – General Left-Recursion Elimination

1. **Order the nonterminals** arbitrarily: $A_1, A_2, \ldots, A_n$.
2. **For** $i = 1$ **to** n**:**
   (a) **Substitute (make direct).** For each $j < i$, replace every production of the form $A_i \to A_j\gamma$ with one copy of $A_i \to \delta\gamma$ for *every* current production $A_j \to \delta$ of A_j – literally pasting A_j's right-hand sides in where A_j used to sit. Because $j < i$, A_j has already been fully processed by this point (its own left recursion, if any, is already gone), so this substitution cannot reintroduce a recursion through A_j – it can only ever convert a recursion *through* an earlier nonterminal into a *direct* recursion on A_i itself.
   (b) **Eliminate (remove it).** Apply §8.5.1's standard transformation to whatever A_i's productions have become after step (a), exactly as if it had been immediately left-recursive from the start.

The two-step shape of the inner loop is the whole idea: **(a) substitute until the recursion is directly visible, (b) then remove it with the one formula we already have.** No new elimination technique is needed – indirect left recursion is only ever a substitution step away from being the immediate case.

Example: Worked Example 1: DB's Classic Two-Nonterminal Case

$$S \to A\texttt{a} \mid \texttt{b} \qquad A \to A\texttt{c} \mid S\texttt{d} \mid \varepsilon$$

Order the nonterminals S ($=A_1$), then A ($=A_2$).
$i = 1$ **(S):** no $j < 1$, so no substitution is possible yet. Checking S's own productions, $S \to A\texttt{a} \mid \texttt{b}$ does not start with S – already free of immediate left recursion. S is left unchanged.
$i = 2$ **(A), step (a) – substitute:** A's production $A \to S\texttt{d}$ starts with the earlier nonterminal S, so replace it with one copy per S-production: $S \to A\texttt{a}$ gives $A\texttt{ad}$, and $S \to \texttt{b}$ gives $\texttt{bd}$. Combined with A's other, unaffected productions ($A\texttt{c}$ and ε, neither of which started with S),

A's production set is now:

$$A \to A\texttt{c} \mid A\texttt{ad} \mid \texttt{bd} \mid \varepsilon$$

This *is* now immediately left-recursive – exactly the point of step (a).
$i = 2$ **(A), step (b) – eliminate:** Matching §8.5.1's template with $\alpha_1 = \texttt{c}$, $\alpha_2 = \texttt{ad}$ (the two A-starting alternatives) and $\beta_1 = \texttt{bd}$, $\beta_2 = \varepsilon$ (the two that don't start with A):

$$A \to \texttt{bd}A' \mid A' \qquad A' \to \texttt{c}A' \mid \texttt{ad}A' \mid \varepsilon$$

Final grammar (left-recursion-free, same language as the original):

$$S \to A\texttt{a} \mid \texttt{b} \qquad A \to \texttt{bd}A' \mid A' \qquad A' \to \texttt{c}A' \mid \texttt{ad}A' \mid \varepsilon$$

Example: Worked Example 2: Mutual Recursion With No ε-Production

A second case, to see the same two steps on a grammar shaped slightly differently – here A and B recurse through each other directly, with no "escape hatch" production like the ε above:

$$A \to B\texttt{a} \mid \texttt{c} \qquad B \to A\texttt{b} \mid \texttt{d}$$

Order A ($=A_1$), then B ($=A_2$).
$i = 1$ **(A):** no $j < 1$; $A \to B\texttt{a} \mid \texttt{c}$ does not start with A – unchanged.
$i = 2$ **(B), step (a) – substitute:** B's production $B \to A\texttt{b}$ starts with the earlier nonterminal A, so substitute A's productions in: $A \to B\texttt{a}$ gives $B\texttt{ab}$, and $A \to \texttt{c}$ gives $\texttt{cb}$. Combined with B's unaffected production ($\texttt{d}$), B's production set becomes:

$$B \to B\texttt{ab} \mid \texttt{cb} \mid \texttt{d}$$

Immediately left-recursive, exactly as intended.
$i = 2$ **(B), step (b) – eliminate:** With $\alpha_1 = \texttt{ab}$ and $\beta_1 = \texttt{cb}$, $\beta_2 = \texttt{d}$:

$$B \to \texttt{cb}B' \mid \texttt{d}B' \qquad B' \to \texttt{ab}B' \mid \varepsilon$$

Final grammar:

$$A \to B\texttt{a} \mid \texttt{c} \qquad B \to \texttt{cb}B' \mid \texttt{d}B' \qquad B' \to \texttt{ab}B' \mid \varepsilon$$

Notice the new nonterminal B' appears *only* because B needed it – substitution alone never by itself requires a new nonterminal; only the elimination step (b) ever introduces one, exactly as in the immediate case.

Exam Focus

Every instance of indirect left-recursion elimination is these same two steps, in this order, never the other way around: **substitute first** (using only already-processed, earlier nonterminals) **to expose direct recursion, then eliminate** it with §8.5.1's one formula. Trying to "eliminate" before the recursion is direct is not meaningful – there is no left-recursive alternative to apply the formula to until substitution has produced one. You are responsible for executing *both* steps correctly on a small (two- or three-nonterminal) example, the way both worked examples above did.

8.6 Self-Check Questions

Practice – Not Graded

1. Given only the algorithm name "CLR(1)," place it correctly on §8.1's tree (top-down or bottom-up? which sub-family?) and state its one-line key feature from memory.
2. Write the 4-tuple (N, T, P, S) explicitly for a small grammar of your own choosing that generates balanced parentheses, $\{a^n b^n \mid n \geq 0\}$-style. (Lecture **??**'s §**??** already showed this language is *not* regular – confirm for yourself that a two-production CFG handles it easily.)
3. Using §8.2's expression grammar, write out the *rightmost* derivation of `id * id + id` (note: different string, different operator order, from the leftmost-derivation example in the lecture).
4. Left-factor the grammar $A \to \texttt{c}A\texttt{d} \mid \texttt{c}B \mid B\texttt{e}$ where the shared prefix is only between the *first two* alternatives. Show your work.
5. Explain, without using the word "slow," exactly what goes wrong – mechanically, in a recursive-descent function – when a grammar is left-recursive. Contrast this precisely with what goes wrong when a grammar merely has unfactored common prefixes.
6. Eliminate immediate left recursion from $S \to S + S \mid S * S \mid \texttt{id}$. (This grammar is also ambiguous – you are not asked to fix that here, only to mechanically apply §8.5.1's transformation and confirm the result is no longer left-recursive.)
7. Apply §8.5.2's general algorithm to $X \to Y\texttt{c} \mid \texttt{e}$, $Y \to X\texttt{d} \mid Y\texttt{f} \mid \texttt{g}$ (order X before Y). Show the substitution step (a) and the resulting immediate-left-recursion elimination step (b) as two clearly separate stages, the way both worked examples in this lecture did – this grammar is deliberately a little richer than either worked example (step (a) here feeds *two* Y-alternatives into step (b), not just one).

Key Takeaways

- Every parsing algorithm in this course is either **top-down** (root → leaves) or **bottom-up** (leaves → root); every bottom-up algorithm covered here is a **shift-reduce**, LR-family algorithm, differing only in how much lookahead its table encodes (SLR(1) ⊊ LALR(1) ⊊ CLR(1) in grammars accepted).
- A **context-free grammar** is exactly the formalism regular expressions cannot provide: the ability to express unbounded, nested structure, via nonterminals and recursive productions.
- A **derivation** rewrites the start symbol one nonterminal at a time down to pure terminals; a **parse tree** is the resulting structure, and many different derivation *orders* (leftmost, rightmost, . . .) can all produce the very same tree.
- **Left factoring** removes shared prefixes between productions so a predictive parser can always decide which production to use with a small, fixed amount of lookahead – skipping it forces either costly backtracking or ever-growing lookahead.
- **Left recursion** is a stricter, correctness-level problem specific to top-down parsing: a left-recursive nonterminal's parsing function recurses before consuming any input, which never terminates. The standard transformation converts left recursion into right recursion without changing the language generated – first for the **immediate** case, then, when needed, via the general algorithm for **indirect** left recursion.
- Left factoring and left recursion are **independent** problems, fixed by **independent** trans-

formations – a grammar can have either, both, or neither, and both must be resolved before any of §8.1's top-down algorithms can be built on top of it.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[ET] Cooper, K. D. and Torczon, L. *Engineering a Compiler*. 2nd ed. Morgan Kaufmann, 2011.

[AP] Appel, A. W. *Modern Compiler Implementation in C/Java/ML*. Cambridge University Press, 2004.